robot_vision

Release 1.2.0

Intel Corporation - Apache-2.0

Jan 07, 2026

CONTENTS:

Algorithms for sensor fusion, environment perception, object detection and tracking in C++ with Python interface

CHAPTER

ONE

ROBOT_VISION.TRACKING

Classes for implementing state filtering and multiple oblect tracking based on the UnscentedKalmanFilter.

TrackedObject	A TrackedObject holds the object's state (position, orientation, velocity, acceleration, size) and properties.
MultiModelKalmanEstimator	Implements the Interacting Multiple Model Kalman Estimator.
MotionModel	MotionModel enum class.
DistanceType	DistanceType enum class.
TrackManagerConfig	Holds all the configuration parameters used by the TrackManager.
TrackManager	Track management system for multiple objects, it holds databases of the current objects on the scene and facilitates updates of multiple objects via uuid queries.
MultipleObjectTracker	Multiple Object Tracking algorithm using the TrackManager in the background.
TrackTracker	Multiple Object Tracking algorithm using the TrackManager in the background.
ClassificationData	Helper class to initialize and get data from a class probability vector (numpy.array).
match	match(tracks: List[robot_vision.extensions.tracking.TrackedObject], measurements: List[robot_vision.extensions.tracking.TrackedObject], distance_type: robot_vision.extensions.tracking.DistanceType = <DistanceType.Spatial: 1>, threshold: float = 1.0) -> Tuple[List[Tuple[int, int]], List[int], List[int]]
mahalanobis_distance	mahalanobis_distance(track: robot_vision.extensions.tracking.TrackedObject, measurement: robot_vision.extensions.tracking.TrackedObject) -> float
association_probability	association_probability(track: robot_vision.extensions.tracking.TrackedObject, measurement: robot_vision.extensions.tracking.TrackedObject) -> float
delta_theta	delta_theta(arg0: float, arg1: float) -> float

1.1 TrackedObject

class robot_vision.tracking.**TrackedObject**

A TrackedObject holds the object's state (position, orientation, velocity, acceleration, size) and properties. It provides interfaces to facilitate its use in state filtering and tracking.

__init__(*self:* robot_vision.extensions.tracking.TrackedObject) → None[1]

Default constructor. The classification probability is initialized as numpy.array([1.0])

add_visual_features(*self:* robot_vision.extensions.tracking.TrackedObject, *arg0:* *numpy.ndarray*[2]*[numpy.float64[m, 1]]*) → None[3]

Set the current visual feature embeddding vector

property age

Time in seconds the object has been updated by a Kalman Estimator.

property association_probability

Returns the last association probability computed during the correction step.

property attributes

only string types are supported.

Type
Dictionary of attributes. Note

property ax

Acceleration component 'x' (forward) in m/s^2.

property ay

Acceleration component 'y' (left) in m/s^2.

property az

Acceleration component 'z' (up) in m/s^2.

property classification

Returns a numpy array with classification probabilities.

property corrected

Returns True if the TrackedObject was the result of a correction step.

property error_covariance

Error covariance matrix, convert to numpy using np.array(tracked_object.error_covariance).

get_visual_features(*self:* robot_vision.extensions.tracking.TrackedObject) → numpy.ndarray[4][numpy.float64[m, 1]]

Get the current visual feature embeddding vector

get_visual_features_matrix(*self:* robot_vision.extensions.tracking.TrackedObject) → numpy.ndarray[5][numpy.float64[m, n]]

Get the current visual feature embeddding vector

property height

Object's height in meters.

isDynamic(*self:* robot_vision.extensions.tracking.TrackedObject) → bool[6]

Returns True if the TrackedObject is considered to be moving.

is_uuid_nil(*self:* robot_vision.extensions.tracking.TrackedObject) → bool[7]

Whether the current uuid is nil/empty.

property jx

Jerk component 'x' (forward) in m/s^3.

property jy

Jerk component 'y' (left) in m/s^3.

property jz

Jerk component 'z' (up) in m/s^3.

property length

Object's length in meters.

property measurement_covariance

Measurement covariance matrix, convert to numpy using np.array(tracked_object.measurement_covariance).

property measurement_covariance_inv

Measurement covariance matrix inverse, convert to numpy using np.array(tracked_object.measurement_covariance_inv).

property measurement_mean

Returns this object's measurement vector as numpy array.

measurement_vector(*self:* robot_vision.extensions.tracking.TrackedObject) → robot_vision.extensions.types.Mat

Measurement vector 7x1, convert to numpy using np.array(tracked_object.measurement_vector).

property predicted_time

Time in seconds the object has been consecutively predicted by a Kalman Estimator.

state_vector(*self:* robot_vision.extensions.tracking.TrackedObject) → robot_vision.extensions.types.Mat

State vector 14x1, convert to numpy using np.array(tracked_object.state_vector).

property tracked_time

Time in seconds the object has been consecutively tracked by a Kalman Estimator.

property uuid

Returns the uuid string.

property vector

Returns this object's state vector as numpy array.

property visual_certainty

Returns the mean of the visual similarity calculation.

property visual_features_set

Read/write access to visual feature set

property vx

Velocity component 'x' (forward) in m/s.

property vy

Velocity component 'y' (left) in m/s.

property vz

Velocity component 'z' (up) in m/s.

property width

Object's width in meters.

property x

Position 'x' in meters.

property y

Position 'y' in meters.

property yaw

Orientation about Z axis in radians.

property yaw_rate

Turn rate about Z axis in radians/s.

property z

Position 'z' in meters.

1.2 MultiModelKalmanEstimator

class robot_vision.tracking.**MultiModelKalmanEstimator**

Implements the Interacting Multiple Model Kalman Estimator. The models can be selected during initialization. The method initialize() must be called before using the KalmanEstimator.

__init__(**args*, ***kwargs*)

Overloaded function.

1. __init__(self: robot_vision.extensions.tracking.MultiModelKalmanEstimator) -> None

Default constructor. The method initialize() must be called before using the KalmanEstimator.

2. __init__(self: robot_vision.extensions.tracking.MultiModelKalmanEstimator, alpha: float = 1.0, beta: float = 1.0) -> None

Initialize with given parameters. The method initialize() must be called before using the KalmanEstimator.

property conditional_probability

Current conditional probability from model a to model b.

correct(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator, *measurement:* robot_vision.extensions.tracking.TrackedObject) → None[8]

Update estimator with current measurement.

current_state(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator) → *robot_vision.extensions.tracking.TrackedObject*

Returns the current filtered state.

current_states(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns the list of internal states.

[1] https://docs.python.org/3/library/constants.html#None
[2] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[3] https://docs.python.org/3/library/constants.html#None
[4] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[5] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[6] https://docs.python.org/3/library/functions.html#bool
[7] https://docs.python.org/3/library/functions.html#bool

initialize(*self: robot_vision.extensions.tracking.MultiModelKalmanEstimator*, *tracked_object: robot_vision.extensions.tracking.TrackedObject*, *timestamp: datetime.datetime*, *process_noise: float = 1e-06*, *measurement_noise: float = 0.0001*, *init_state_covariance: float = 1.0*, *motion_models: List[rv::tracking::MotionModelType] = []*) → None[9]

Initialize the MultiModelKalmanEstimator with the given tracked object.

kalman_filter_error_covariance(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator, *n: int*[10]) → robot_vision.extensions.types.Mat

Get error covariance of the Nth kalman filter.

kalman_filter_measurement_covariance(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator, *n: int*[11]) → robot_vision.extensions.types.Mat

Get measurement covariance of the Nth kalman filter.

property model_probability

Probability of following certain motion model.

predict(**args*, ***kwargs*)

Overloaded function.

1. predict(self: robot_vision.extensions.tracking.MultiModelKalmanEstimator, deltaT: float) -> None

Predict the position at T+deltaT time.

2. predict(self: robot_vision.extensions.tracking.MultiModelKalmanEstimator, timestamp: datetime.datetime) -> None

Predict the position at given timestamp.

timestamp(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator) → datetime.datetime[12]

Read current timestamp.

track(*self:* robot_vision.extensions.tracking.MultiModelKalmanEstimator, *measurement:* robot_vision.extensions.tracking.TrackedObject, *timestamp: datetime.datetime*[13]) → None[14]

Trigger the track step for the next timestamp.

property transition_probability

Transition probability from model a to model b.

1.3 MotionModel

class robot_vision.tracking.**MotionModel**

MotionModel enum class.

Members:

CV : Constant Velocity.

CA : Constant Acceleration.

CP : Constant Position.

[8] https://docs.python.org/3/library/constants.html#None
[9] https://docs.python.org/3/library/constants.html#None
[10] https://docs.python.org/3/library/functions.html#int
[11] https://docs.python.org/3/library/functions.html#int
[12] https://docs.python.org/3/library/datetime.html#datetime.datetime
[13] https://docs.python.org/3/library/datetime.html#datetime.datetime
[14] https://docs.python.org/3/library/constants.html#None

CJ : Constant Jerk.

CTRV : Constant Turn-Rate and Velocity.

__init__(*self:* robot_vision.extensions.tracking.MotionModel, *value:* *int*[15]) → None[16]

property name

1.4 DistanceType

class robot_vision.tracking.**DistanceType**

DistanceType enum class.

Members:

Visual : Use the cosine distance on the visual features.

Spatial : Standard euclidean distance that considers the object position (x,y,z).

VisualSpatial : Scaled euclidean metric distance. It is scaled by visual distance between objects.

VisualMultiClass : The sum of the multiclass probability conflict and the visual distance

SpatialMultiClass : Scaled euclidean metric distance. It is scaled by the multiclass probability difference.

VisualSpatialMultiClass : Scaled euclidean metric distance. It is scaled by the sum of the multiclass probability conflict and the visual distance.

Mahalanobis : Mahalanobis distance that considers the objects measurement vector.

__init__(*self:* robot_vision.extensions.tracking.DistanceType, *value:* *int*[17]) → None[18]

property name

1.5 TrackManagerConfig

class robot_vision.tracking.**TrackManagerConfig**

Holds all the configuration parameters used by the TrackManager.

__init__(*self:* robot_vision.extensions.tracking.TrackManagerConfig) → None[19]

Initialize TrackManagerConfig with default parameters.

property default_measurement_noise

Default measurement noise passed to the KalmanEstimator init function.

property default_process_noise

Default process noise passed to the KalmanEstimator init function.

property init_state_covariance

Default init state covariance passed to the KalmanEstimator init function.

property max_number_of_unreliable_frames

Number of frames to measure an object before considering it a reliable object.

[15] https://docs.python.org/3/library/functions.html#int
[16] https://docs.python.org/3/library/constants.html#None
[17] https://docs.python.org/3/library/functions.html#int
[18] https://docs.python.org/3/library/constants.html#None

property motion_models

List of motion models to use. It defaults to [CV, CA, CTRV]

property non_measurement_frames_dynamic

Sets the maximum number of non measurement frames for a dynamic object. The track will be removed if it is not seen for given number of frames.

property non_measurement_frames_static

Sets the maximum number of non measurement frames for a static object. The track will be suspended if it is not seen for given number of frames.

property reactivation_frames

Number of frames to measure a suspended object before reactivating the track.

1.6 TrackManager

class robot_vision.tracking.**TrackManager**

Track management system for multiple objects, it holds databases of the current objects on the scene and facilitates updates of multiple objects via uuid queries.

__init__(**args*, ***kwargs*)

Overloaded function.

1. __init__(self: robot_vision.extensions.tracking.TrackManager) -> None

Construct with default config

2. __init__(self: robot_vision.extensions.tracking.TrackManager, track_manager_config: robot_vision.extensions.tracking.TrackManagerConfig) -> None

Construct with given config

3. __init__(self: robot_vision.extensions.tracking.TrackManager, auto_id_generation: bool) -> None

Construct with default config. Set auto_id_generation to False to use the already assigned track_id instead.

4. __init__(self: robot_vision.extensions.tracking.TrackManager, track_manager_config: robot_vision.extensions.tracking.TrackManagerConfig, auto_id_generation: bool) -> None

Construct with default config. Set auto_id_generation to False to use the already assigned track_id instead.

property config

Current track manager configuration

correct(*self:* robot_vision.extensions.tracking.TrackManager) → None[20]

Trigger state correction for all tracks.

create_track(*self:* robot_vision.extensions.tracking.TrackManager, *object:* robot_vision.extensions.tracking.TrackedObject, *timestamp:* *datetime.datetime*[21]) → uuid

Create a new track, returns object uuid of new track.

delete_track(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → None[22]

Delete the given track uuid from the track manager.

get_drifting_tracks(*self:* robot_vision.extensions.tracking.TrackManager) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of tracks in risk of being deleted. Objects that have not been visible for config.non_measurement_frames_dynamic / 2.

[19] https://docs.python.org/3/library/constants.html#None

get_kalman_estimator(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → *robot_vision.extensions.tracking.MultiModelKalmanEstimator*

Returns the MultiModelKalmanEstimator stored for the given uuid.

get_reliable_tracks(*self:* robot_vision.extensions.tracking.TrackManager) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of all tracks classified as reliable.

get_suspended_tracks(*self:* robot_vision.extensions.tracking.TrackManager) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of suspended tracks. Static objects that have not been visible for config.non_measurement_frames_static frames.

get_track(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → *robot_vision.extensions.tracking.TrackedObject*

Returns the TrackedObject stored for the given uuid.

get_tracks(*self:* robot_vision.extensions.tracking.TrackManager) → List[*robot_vision.extensions.tracking.TrackedObject*]

returns a list of all active tracks.

get_unreliable_tracks(*self:* robot_vision.extensions.tracking.TrackManager) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of all tracks classified as unreliable.

has_uuid(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → bool[23]

Check wether the given Id is registered in the track manager.

is_reliable(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → bool[24]

Check whether the given track uuid is reliable.

is_suspended(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → bool[25]

Check whether the given track uuid is suspended.

predict(**args*, ***kwargs*)

Overloaded function.

1. predict(self: robot_vision.extensions.tracking.TrackManager, deltaT: float) -> None

Predict at T+deltaT time.

2. predict(self: robot_vision.extensions.tracking.TrackManager, timestamp: datetime.datetime) -> None

Predict at the given timestamp.

reactivate_track(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → None[26]

Move a suspended track uuid to active tracks.

set_measurement(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*, *measurement:* robot_vision.extensions.tracking.TrackedObject) → None[27]

Create a new track, returns object uuid of new track.

suspend_track(*self:* robot_vision.extensions.tracking.TrackManager, *uuid: uuid*) → None[28]

Set the given track uuid as suspended.

1.7 MultipleObjectTracker

class robot_vision.tracking.**MultipleObjectTracker**

Multiple Object Tracking algorithm using the TrackManager in the background. It performs an association step using the Gated Hungarian matcher.

__init__(**args*, ***kwargs*)

Overloaded function.

1. __init__(self: robot_vision.extensions.tracking.MultipleObjectTracker) -> None

Default constructor, use default config parameters.

2. __init__(self: robot_vision.extensions.tracking.MultipleObjectTracker, track_manager_config: robot_vision.extensions.tracking.TrackManagerConfig) -> None

Use the given config parameters for the track manager.

3. __init__(self: robot_vision.extensions.tracking.MultipleObjectTracker, track_manager_config: robot_vision.extensions.tracking.TrackManagerConfig, distance_type: robot_vision.extensions.tracking.DistanceType, distance_threshold: float) -> None

Use the given config parameters for the track manager. Initialize with the given distance type and threshold.

get_reliable_tracks(*self:* robot_vision.extensions.tracking.MultipleObjectTracker) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of all active reliable tracks.

get_tracks(*self:* robot_vision.extensions.tracking.MultipleObjectTracker) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of all active tracks

timestamp(*self:* robot_vision.extensions.tracking.MultipleObjectTracker) → datetime.datetime[29]

Read current timestamp.

track(**args*, ***kwargs*)

Overloaded function.

1. track(self: robot_vision.extensions.tracking.MultipleObjectTracker, objects: List[robot_vision.extensions.tracking.TrackedObject], timestamp: datetime.datetime, probability_threshold: float = 0.5) -> None

Trigger the track step for the next timestamp. Use the default distance type and threshold.

2. track(self: robot_vision.extensions.tracking.MultipleObjectTracker, objects: List[robot_vision.extensions.tracking.TrackedObject], timestamp: datetime.datetime, distance_type: robot_vision.extensions.tracking.DistanceType, distance_threshold: float, probability_threshold: float = 0.5) -> None

Trigger the track step for the next timestamp. Run match() with the given distance type and threshold.

[20] https://docs.python.org/3/library/constants.html#None
[21] https://docs.python.org/3/library/datetime.html#datetime.datetime
[22] https://docs.python.org/3/library/constants.html#None
[23] https://docs.python.org/3/library/functions.html#bool
[24] https://docs.python.org/3/library/functions.html#bool
[25] https://docs.python.org/3/library/functions.html#bool
[26] https://docs.python.org/3/library/constants.html#None
[27] https://docs.python.org/3/library/constants.html#None
[28] https://docs.python.org/3/library/constants.html#None
[29] https://docs.python.org/3/library/datetime.html#datetime.datetime

1.8 TrackTracker

class robot_vision.tracking.**TrackTracker**

Multiple Object Tracking algorithm using the TrackManager in the background. This tracker does not perform any association step, instead it relies on the object's uuid for association.

__init__(**args*, ***kwargs*)

Overloaded function.

1. __init__(self: robot_vision.extensions.tracking.TrackTracker) -> None

Default constructor, use default config parameters.

2. __init__(self: robot_vision.extensions.tracking.TrackTracker, track_manager_config: robot_vision.extensions.tracking.TrackManagerConfig) -> None

Use the given config parameters for the track manager.

get_reliable_tracks(*self:* robot_vision.extensions.tracking.TrackTracker) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of all active reliable tracks.

get_tracks(*self:* robot_vision.extensions.tracking.TrackTracker) → List[*robot_vision.extensions.tracking.TrackedObject*]

Returns a list of all active tracks.

timestamp(*self:* robot_vision.extensions.tracking.TrackTracker) → datetime.datetime[30]

Read current timestamp.

track(*self:* robot_vision.extensions.tracking.TrackTracker, *tracked_objects:* *List[*robot_vision.extensions.tracking.TrackedObject*]*, *timestamp:* *datetime.datetime*[31]) → None[32]

Trigger the track step for the next timestamp. Note: The objects must have an uuid already assigned.

1.9 ClassificationData

class robot_vision.tracking.**ClassificationData**

Helper class to initialize and get data from a class probability vector (numpy.array).

__init__(**args*, ***kwargs*)

Overloaded function.

1. __init__(self: robot_vision.extensions.tracking.ClassificationData) -> None

Default constructor. The classes vector will default to ['Unknown'].

2. __init__(self: robot_vision.extensions.tracking.ClassificationData, classes: List[str]) -> None

Initialize ClassificationData with the given class list

property classes

List of classes defined in this ClassificationData.

classification(*self:* *robot_vision.extensions.tracking.ClassificationData*, *class:* *str*, *probability:* *float* = *1.0*) → numpy.ndarray[33][numpy.float64[m, 1]]

Create a classification vector with the given class set to the corresponding probability.

[30] https://docs.python.org/3/library/datetime.html#datetime.datetime
[31] https://docs.python.org/3/library/datetime.html#datetime.datetime
[32] https://docs.python.org/3/library/constants.html#None

get_class(*self:* robot_vision.extensions.tracking.ClassificationData, *classification:* *numpy.ndarray*[34]*[numpy.float64[m, 1]]*) → str[35]

Returns the name of the class with the maximum probability.

get_class_index(*self:* robot_vision.extensions.tracking.ClassificationData, *class_name: str*[36]) → int[37]

Returns the index for the given class name.

prior(*self:* robot_vision.extensions.tracking.ClassificationData) → numpy.ndarray[38][numpy.float64[m, 1]]

Generate a prior classification assigning the same probability for all clases.

unform_prior(*self:* robot_vision.extensions.tracking.ClassificationData, *probability: float*[39]) → numpy.ndarray[40][numpy.float64[m, 1]]

Generate a prior vector using the given probability for each class.

1.10 robot_vision.tracking.match

robot_vision.tracking.**match**(*tracks: List[robot_vision.extensions.tracking.TrackedObject], measurements: List[robot_vision.extensions.tracking.TrackedObject], distance_type: robot_vision.extensions.tracking.DistanceType = <DistanceType.Spatial: 1>, threshold: float = 1.0*) → Tuple[List[Tuple[int[41], int[42]]], List[int[43]], List[int[44]]]

Match measurements to tracks. Returns a tuple containing (track and object index, unassigned tracks, unassigned objects).

1.11 robot_vision.tracking.mahalanobis_distance

robot_vision.tracking.**mahalanobis_distance**(*track:* robot_vision.extensions.tracking.TrackedObject, *measurement:* robot_vision.extensions.tracking.TrackedObject) → float[45]

Calculates the mahalanobis distance between the predicted measurement of the track and the new measurement

1.12 robot_vision.tracking.association_probability

robot_vision.tracking.**association_probability**(*track:* robot_vision.extensions.tracking.TrackedObject, *measurement:* robot_vision.extensions.tracking.TrackedObject) → float[46]

Calculates the association probability between the predicted measurement of the track and the new measurement

[33] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[34] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[35] https://docs.python.org/3/library/stdtypes.html#str
[36] https://docs.python.org/3/library/stdtypes.html#str
[37] https://docs.python.org/3/library/functions.html#int
[38] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[39] https://docs.python.org/3/library/functions.html#float
[40] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[41] https://docs.python.org/3/library/functions.html#int
[42] https://docs.python.org/3/library/functions.html#int
[43] https://docs.python.org/3/library/functions.html#int
[44] https://docs.python.org/3/library/functions.html#int
[45] https://docs.python.org/3/library/functions.html#float
[46] https://docs.python.org/3/library/functions.html#float

1.13 robot_vision.tracking.delta_theta

robot_vision.tracking.**delta_theta**(*arg0: float*[47], *arg1: float*[48]) → float[49]

Calculate the difference between two angles, considering possible jumps of pi.

[47] https://docs.python.org/3/library/functions.html#float
[48] https://docs.python.org/3/library/functions.html#float
[49] https://docs.python.org/3/library/functions.html#float

CHAPTER

TWO

ROBOT_VISION.TRACKING.CLASSIFICATION

Classification probability related functions

distance	distance(classification_a: numpy.ndarray[numpy.float64[m, 1]], classification_b: numpy.ndarray[numpy.float64[m, 1]]) -> float
similarity	similarity(classification_a: numpy.ndarray[numpy.float64[m, 1]], classification_b: numpy.ndarray[numpy.float64[m, 1]]) -> float
combine	combine(classification_a: numpy.ndarray[numpy.float64[m, 1]], classification_b: numpy.ndarray[numpy.float64[m, 1]]) -> numpy.ndarray[numpy.float64[m, 1]]

robot_vision.tracking.classification.**distance**(*classification_a: numpy.ndarray[50][numpy.float64[m, 1]]*, *classification_b: numpy.ndarray[51][numpy.float64[m, 1]]*) → float[52]

Calculate the distance between two classification probability vectors.

robot_vision.tracking.classification.**similarity**(*classification_a: numpy.ndarray[53][numpy.float64[m, 1]]*, *classification_b: numpy.ndarray[54][numpy.float64[m, 1]]*) → float[55]

Calculate how similar two given classifications are.

robot_vision.tracking.classification.**combine**(*classification_a: numpy.ndarray[56][numpy.float64[m, 1]]*, *classification_b: numpy.ndarray[57][numpy.float64[m, 1]]*) → numpy.ndarray[58][numpy.float64[m, 1]]

Combine probability vectors using multiclass bayes update.

[50] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[51] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[52] https://docs.python.org/3/library/functions.html#float
[53] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[54] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[55] https://docs.python.org/3/library/functions.html#float
[56] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[57] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray
[58] https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray

CHAPTER

THREE

ROBOT_VISION.TRACKING.VISUAL

Visual features (ReID) related functions

distance	distance(*args, **kwargs) Overloaded function.
similarity	similarity(*args, **kwargs) Overloaded function.

robot_vision.tracking.visual.**distance**(**args*, ***kwargs*)

Overloaded function.

1. distance(visual_features_a: numpy.ndarray[numpy.float64[m, 1]], visual_features_b: numpy.ndarray[numpy.float64[m, 1]]) -> float

Calculate the visual distance (cosine distance) between two visual vectors.

2. distance(visual_features: numpy.ndarray[numpy.float64[m, 1]], visual_features_matrix: numpy.ndarray[numpy.float64[m, n]]) -> numpy.ndarray[numpy.float64[m, 1]]

Calculate the visual distance (cosine distance) between two visual vectors.

robot_vision.tracking.visual.**similarity**(**args*, ***kwargs*)

Overloaded function.

1. similarity(visual_features_a: numpy.ndarray[numpy.float64[m, 1]], visual_features_b: numpy.ndarray[numpy.float64[m, 1]]) -> float

Calculate the visual similarity (1 - cosine distance) between two visual vectors.

2. similarity(visual_features: numpy.ndarray[numpy.float64[m, 1]], visual_features_matrix: numpy.ndarray[numpy.float64[m, n]]) -> numpy.ndarray[numpy.float64[m, 1]]

Calculate the visual similarity (1 - cosine distance) between a visual vector and a matrix of visual vectors.

INDEX

Symbols

A

C

D

E

G

H

I

J

K

L

M

N

P

R

S

T

U

V

W

X

Y

Z